

Introductory Econometrics with Recitation: TA Session Week 6

Danny Po-Hsien Kang (康柏賢)

March 31, 2026

Today's agenda

1 Data Wrangling

- `nrow()`, `ncol()`, `dim()`
- `nchar()`
- `substring()`
- `grep()`

2 Summary Statistics Table

- `table()`
- `addmargins()`

3 Multiple Regression

- `lm_robust()`
- `screenreg()`

Number of rows/columns: `nrow()`, `ncol()`, `dim()`

- **Syntax:**

```
nrow(df) # or matrix  
ncol(df)  
dim(df)
```

- These functions work on dataframes or matrices, but not on vectors.

- **Example:**

```
df <- data.frame(  
  name  = c("Alice", "Bob"),  
  score = c(80, 90),  
  pass  = c(TRUE, TRUE)  
)  
  
nrow(df) # 2  
ncol(df) # 3  
dim(df)  # 2 3
```

Count Number of Characters: nchar()

- `nchar()` counts the number of characters in a string.
- **Syntax:**

```
nchar(x)
```

- **Example:**

```
word1 <- "apple"  
nchar(word1)  
# [1] 5  
  
words <- nchar(c(15, -3.5, TRUE))  
nchar(words)  
# [1] 2 4 1
```

Trim String: substring()

- **Syntax:**

```
substring(str, start_index, end_index)
```

- **Example:**

```
str1 <- "Cat_and_Dog"  
substr1 <- substring(str1, 1, 3)  
print(substr1)
```

- **Output:**

```
[1] "Cat"
```

Find patterns inside element: `grepl()`

- **Syntax:**

```
grepl(pattern, str)
grepl(pattern, str_vec)
```

- `grepl()` will return a logical vector: TRUE if the pattern is inside that element, FALSE otherwise.
- **Example:**

```
grepl("app", "apple") # TRUE

fruits <- c("apple", "banana", "pineapple", "pear")
grepl("apple", fruits) # TRUE FALSE TRUE FALSE

strings <- c("abcba", "acbc b", "abbbc", "babcc")
grepl("abc", strings) # TRUE FALSE FALSE TRUE
```

Create Contingency Table: table()

application type	home ownership		
	Rent	Mortgage	Own
individual	3496	3839	1170
joint	362	950	183

Table: Homeownership by application type

Create Contingency Table: table()

- **Syntax:**

```
# dnn = dimension names  
A <- table(..., dnn = ...)
```

- **Example:**

```
loan <- read.csv("loans_full_schema.csv")  
table <- table(loan$application_type,  
               loan$homeownership,  
               dnn = c("application type",  
                       "home ownership"  
                       )  
               )
```


Add Sum Row and Sum Column: addmargins()

application_type	homeownership			
	Rent	Mortgage	Own	Total
individual	3496	3839	1170	8505
joint	362	950	183	1495
Total	3858	4789	1353	10 000

Table: Homeownership by application type, with sum row and sum column

Add Sum Row and Sum Column: addmargins()

- **Syntax:**

```
A <- addmargins(table)
```

- **Example:**

```
table_withsum <- addmargins(table)
```

Heteroskedasticity-Robust Standard Errors: `lm_robust()`

- **Homoskedasticity** (A5) is often violated, that is,

$$\text{Var}(u_i | X_i) \text{ depends on } X_i$$

- $\hat{\beta}_1$ is still unbiased and consistent; however, when estimating $\text{Var}(\hat{\beta}_1)$, we need a heteroskedasticity-robust estimator:

$$\hat{\sigma}_{\hat{\beta}_1}^2 = \frac{\frac{1}{n} \times \frac{1}{n-2} \sum_{i=1}^n (X_i - \bar{X})^2 \hat{u}_i^2}{\left[\frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2 \right]^2}$$

- This can be implemented by the `lm_robust()` function.
- No need to memorize the formula, just interpret it.

Heteroskedasticity-Robust Standard Errors: `lm_robust()`

- **Syntax:**

```
library(estimatr) # import the package!

lm_model_robust <- lm_robust(
  data = myData,
  Y ~ X1 + X2 + X3
)
```

- **Example:**

```
lm_model_multiple_robust <- lm_robust(
  data = birth,
  weight ~ weeks + gender + visits
)
```

Heteroskedasticity-Robust Standard Errors: `lm_robust()`

• Output:

```
> summary(lm_model_multiple_robust)
```

Call:

```
lm_robust(formula = weight ~ weeks + gender + visits, data = birth)
```

Standard error type: HC2

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	CI Lower	CI Upper	DF
(Intercept)	-6.208828	0.532882	-11.651	1.745e-29	-7.25454	-5.16312	986
weeks	0.339617	0.013970	24.311	1.163e-102	0.31220	0.36703	986
gendermale	0.365473	0.070477	5.186	2.611e-07	0.22717	0.50377	986
visits	0.009322	0.009229	1.010	3.127e-01	-0.00879	0.02743	986

Multiple R-squared: 0.4492 , Adjusted R-squared: 0.4476

F-statistic: 216.8 on 3 and 986 DF, p-value: < 2.2e-16

Heteroskedasticity-Robust Standard Errors: lm_robust()

• Output using lm():

```
> summary(lm_model_multiple)
```

Call:

```
lm(formula = weight ~ weeks + gender + visits, data = birth)
```

Residuals:

Min	1Q	Median	3Q	Max
-3.7393	-0.6886	-0.0053	0.6802	4.2551

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-6.208828	0.474458	-13.086	< 2e-16 ***
weeks	0.339617	0.012485	27.201	< 2e-16 ***
gendermale	0.365473	0.070411	5.191	2.55e-07 ***
visits	0.009322	0.009045	1.031	0.303

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.107 on 986 degrees of freedom

Multiple R-squared: 0.4492, Adjusted R-squared: 0.4476

F-statistic: 268.1 on 3 and 986 DF, p-value: < 2.2e-16

Regression Table: screenreg()

- **Syntax:**

```
library(texreg) # import the package!

screenreg(
  list(model1, model2, ...),
  custom.model.names =
    c("Model 1", "Model 2", ...),
  include.ci = FALSE,
  digit = 3
)
```

Regression Table: screenreg()

- **Example:**

```
screenreg(  
  # models you want to present in table  
  list(lm_model, lm_model_multiple, lm_model_multiple_robust),  
  
  # names for each model  
  custom.model.names =  
    c("Model 1: Single",  
      "Model 2: Multiple",  
      "Model 3: Multiple (Robust SE)"),  
  
  # present standard error  
  include.ci = FALSE,  
  digit = 3  
)
```


Regression Table: screenreg()

Result

	Model 1: Single	Model 2: Multiple	Model 3: Multiple (Robust SE)
(Intercept)	-6.095 *** (0.465)	-6.209 *** (0.474)	-6.209 *** (0.533)
weeks	0.344 *** (0.012)	0.340 *** (0.012)	0.340 *** (0.014)
gendermale		0.365 *** (0.070)	0.365 *** (0.070)
visits		0.009 (0.009)	0.009 (0.009)
R ²	0.449	0.449	0.449
Adj. R ²	0.448	0.448	0.448
Num. obs.	998	990	990
RMSE			1.107
*** p < 0.001; ** p < 0.01; * p < 0.05			